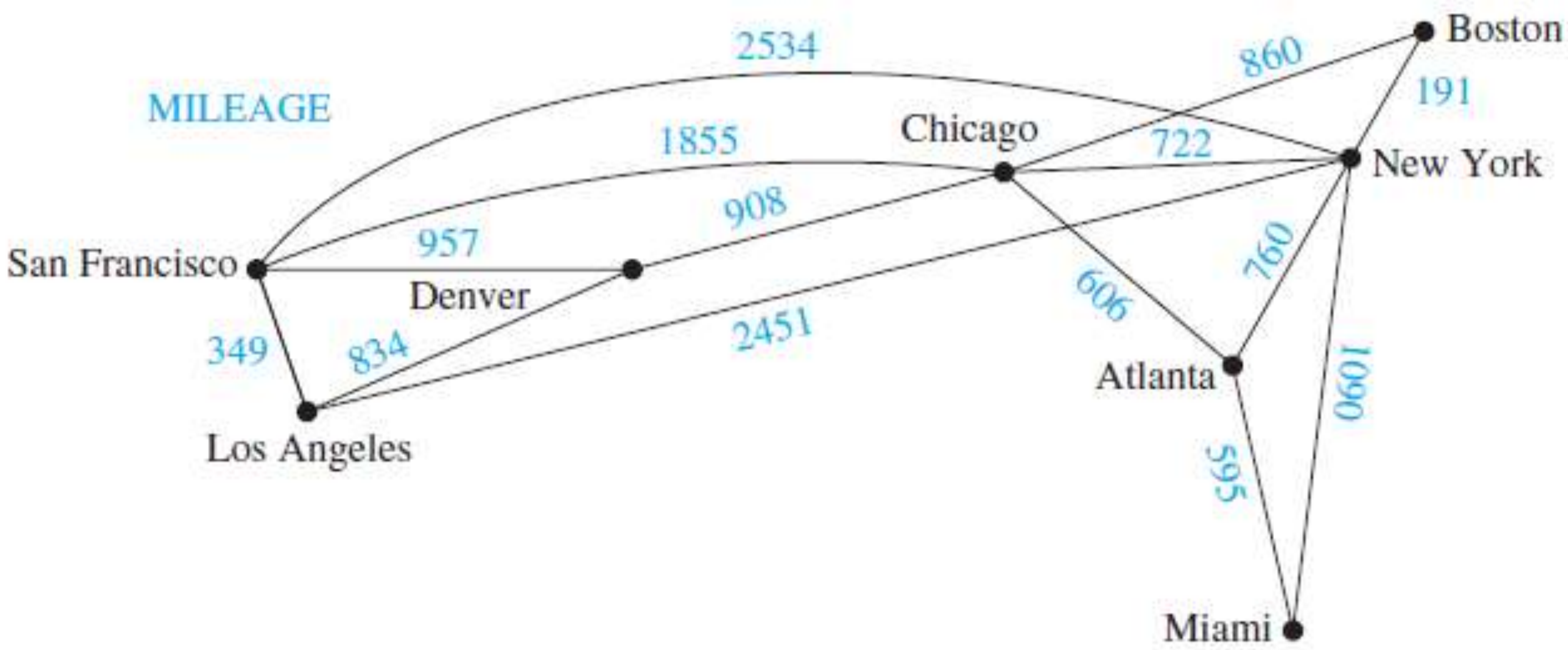
Shortest Path Algorithm

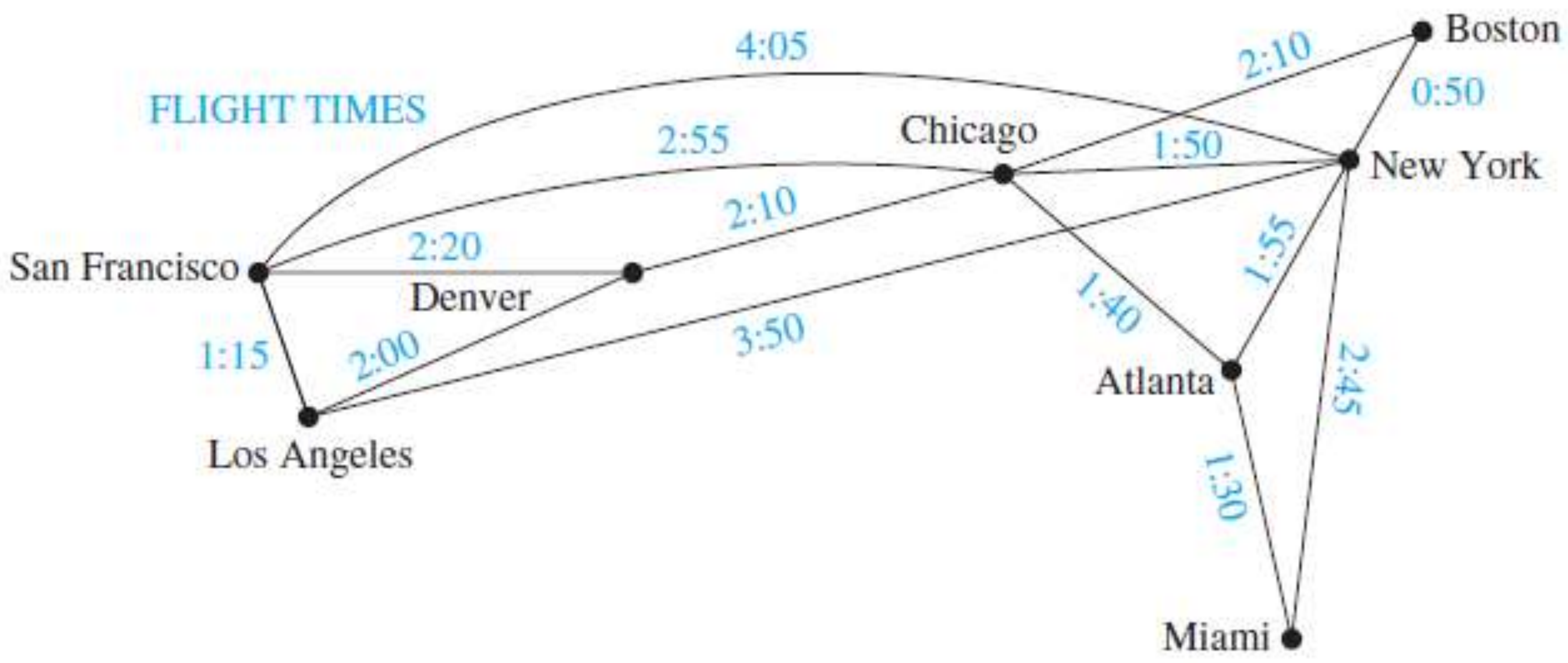
Introduction

- cities by vertices and flights by edges
- Graphs that have a number assigned to each edge are called **weighted graphs**.
- Weighted graphs are used to model computer networks
- Let us take a example of **Weighted Graphs Modeling an Airline System**
- Problems involving distances can be modeled by assigning distances between cities to the edges.
- Problems involving flight time can be modeled by assigning flight times to edges.
- Problems involving fares can be modeled by assigning fares to the edges.

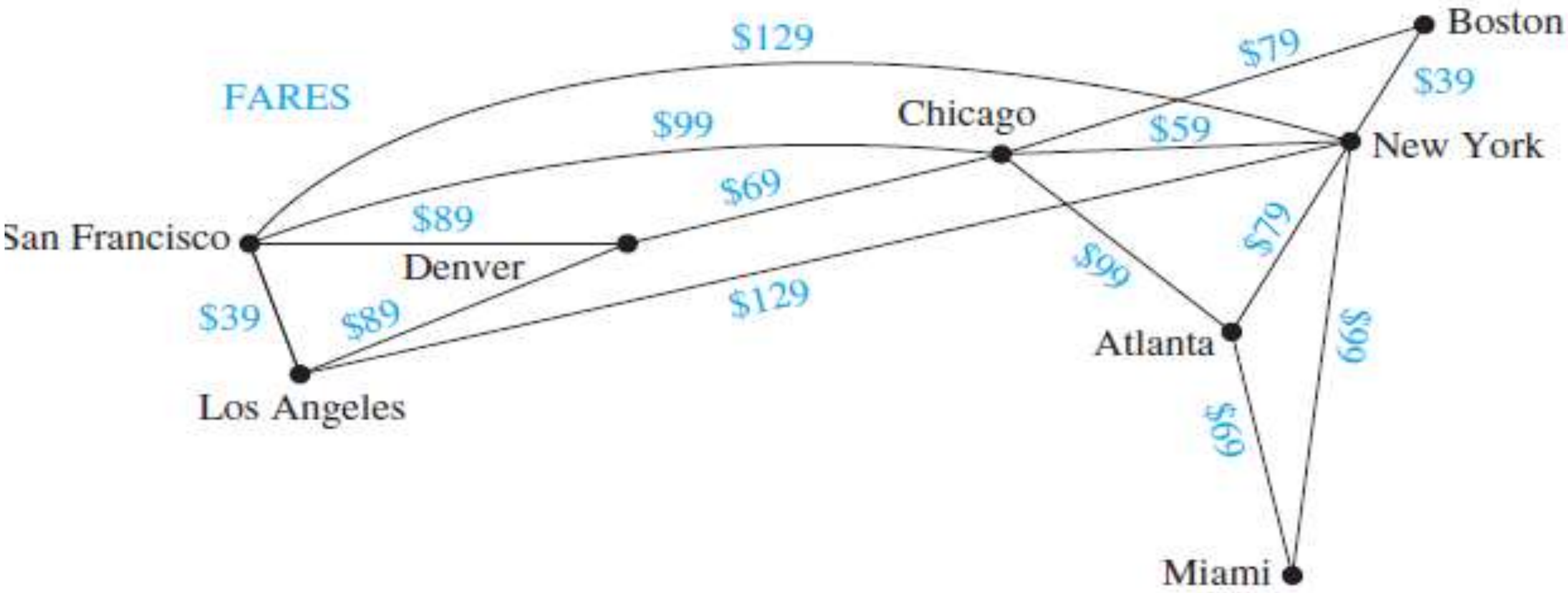
Weighted graph: Mileage



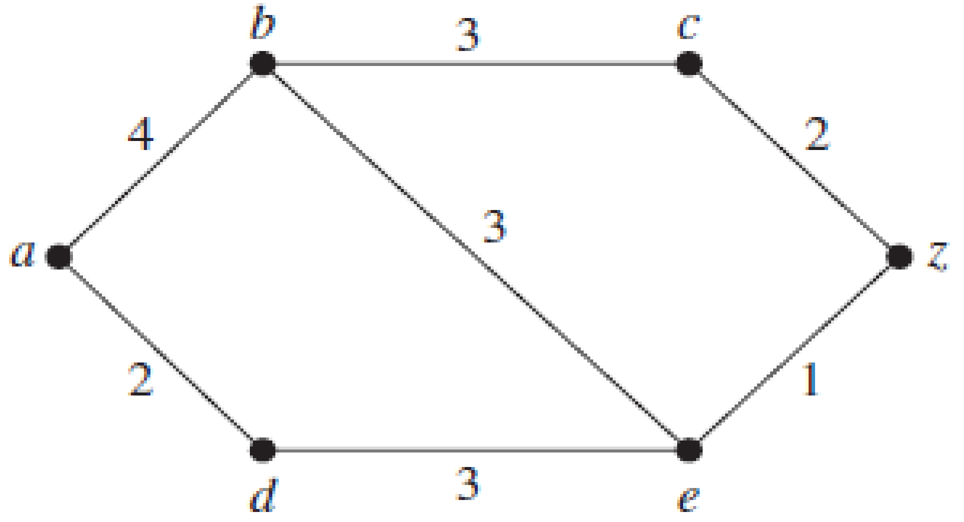
Weighted graph: Flight Times



Weighted graph: Fares



What is the length of a shortest path between *a* and *z* in the weighted graph shown in Figure ?



The shortest path algorithm

- The length of a path in a weighted graph is the sum of weights of the edges of this path and the shortest path between the two vertices is the minimum length of the path.

Dijkstra’s Algorithm

Step 1: label the initial vertex of the graph with weight zero.

Step 2 : calculate the weights of all vertices adjacent to initial vertex corresponding to the weights of the edges incident on the initial vertex.

Step 3 : label these vertices with smallest possible value of their weights .

Step 4 : calculate the weights of all those vertices with are adjacent to the vertices with minimum weights determined in step 3

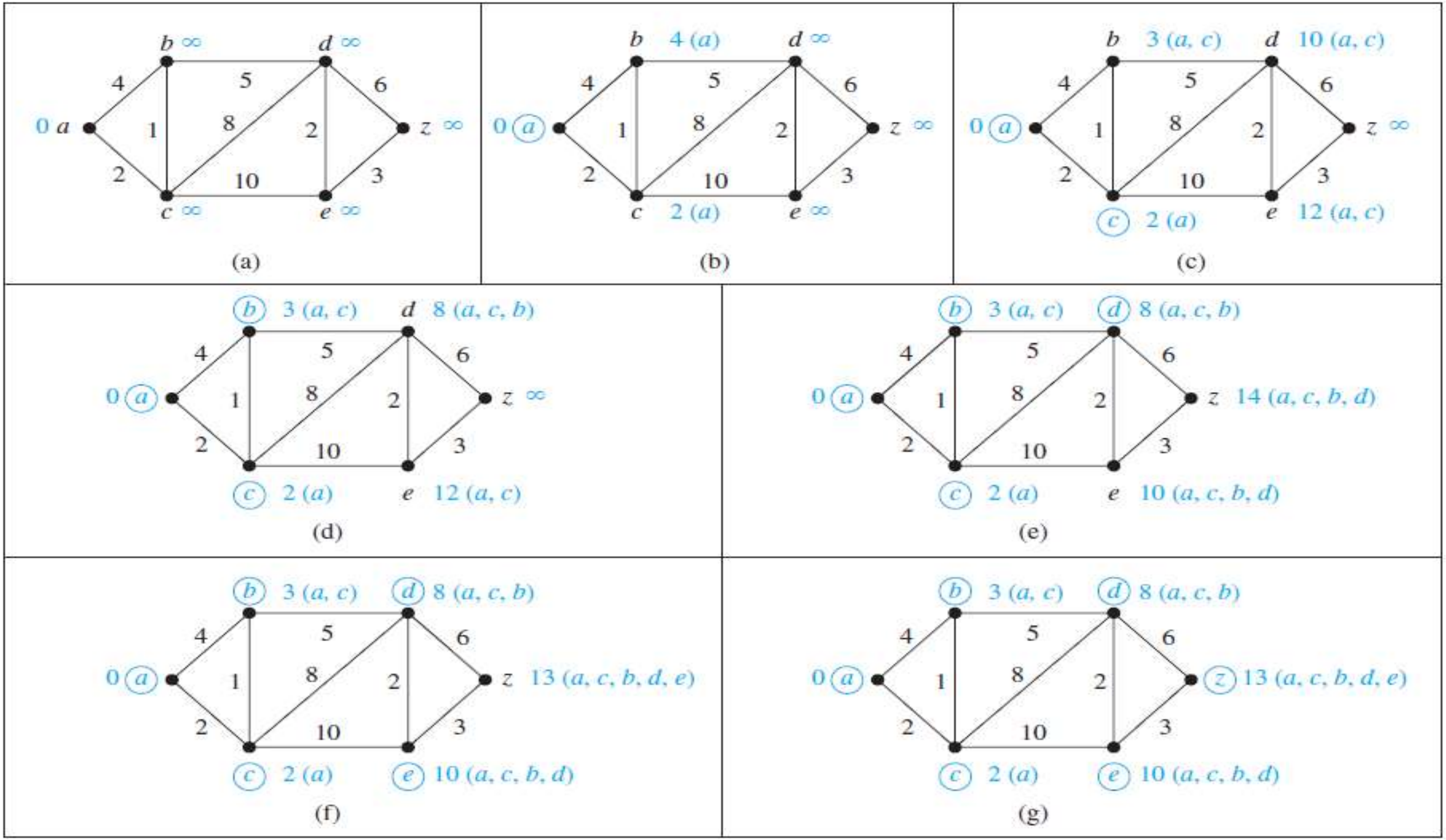
Step 5 : label these vertices with minimum weight.

Step 6: continue this process until all the vertices of weighted graph are labeled.

Step 7: trace the path of cumulative minimum weight from initial vertex to desire vertex.

ALGORITHM 1 Dijkstra's Algorithm.

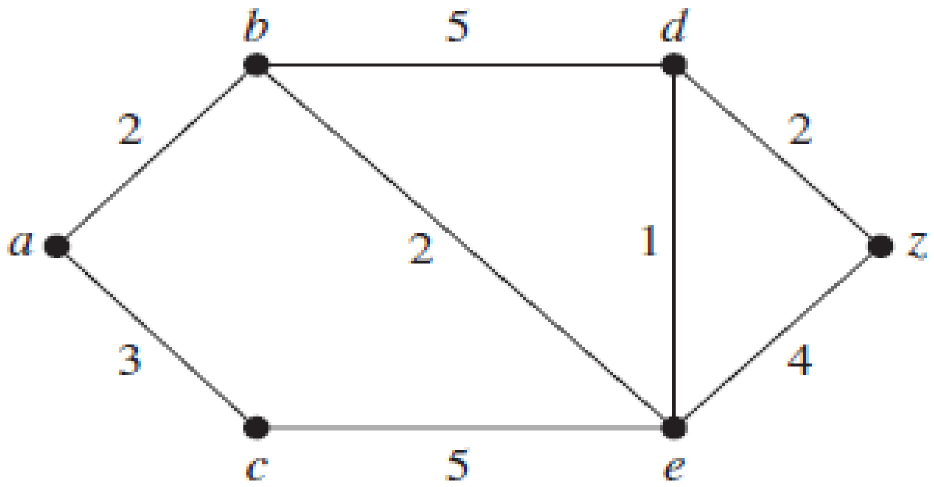
```
procedure Dijkstra(G: weighted connected simple graph, with
        all weights positive)
{G has vertices a = v_0, v_1, ..., v_n = z and lengths w(v_i, v_j)
        where w(v_i, v_j) = ∞ if {v_i, v_j} is not an edge in G}
for i := 1 to n
        L(v_i) := ∞
L(a) := 0
S := ∅
{the labels are now initialized so that the label of a is 0 and all
        other labels are ∞, and S is the empty set}
while z ∉ S
        u := a vertex not in S with L(u) minimal
        S := S ∪ {u}
        for all vertices v not in S
                if L(u) + w(u, v) < L(v) then L(v) := L(u) + w(u, v)
                {this adds a vertex to S with minimal label and updates the
                labels of vertices not in S}
return L(z) {L(z) = length of a shortest path from a to z}
```



b ∞
d ∞
4
5
6
0 a
1
8
2
z ∞
2
10
3
c ∞
e ∞
(a)
b 4 (a)
d ∞
0 a
z ∞
c 2 (a)
e ∞
(b)
b 3 (a, c)
d 10 (a, c)
0 a
z ∞
c 2 (a)
e 12 (a, c)
(c)
b 3 (a, c)
d 8 (a, c, b)
0 a
z ∞
c 2 (a)
e 12 (a, c)
(d)
b 3 (a, c)
d 8 (a, c, b)
0 a
z 14 (a, c, b, d)
c 2 (a)
e 10 (a, c, b, d)
(e)
b 3 (a, c)
d 8 (a, c, b)
0 a
z 13 (a, c, b, d, e)
c 2 (a)
e 10 (a, c, b, d)
(f)
b 3 (a, c)
d 8 (a, c, b)
0 a
z 13 (a, c, b, d, e)
c 2 (a)
e 10 (a, c, b, d)
(g)

Classwork

- Find the shortest path of the given figure.



The Travelling Salesman Problem

- A traveling salesperson wants to visit each of *n* cities exactly once and return to his starting point.
- suppose that the salesperson wants to visit Detroit, Toledo, Saginaw, Grand Rapids, and Kalamazoo.
- In which order should he visit these cities to travel the total distance?
- To solve this problem we can assume the salesperson starts in Detroit and examine all possible ways for him to visit the other four cities and then return to Detroit.
- There are a total of 24 such circuits, but because we travel the same distance when we travel a circuit in reverse order, we need only consider 12 different circuits to find the minimum total distance he must travel.

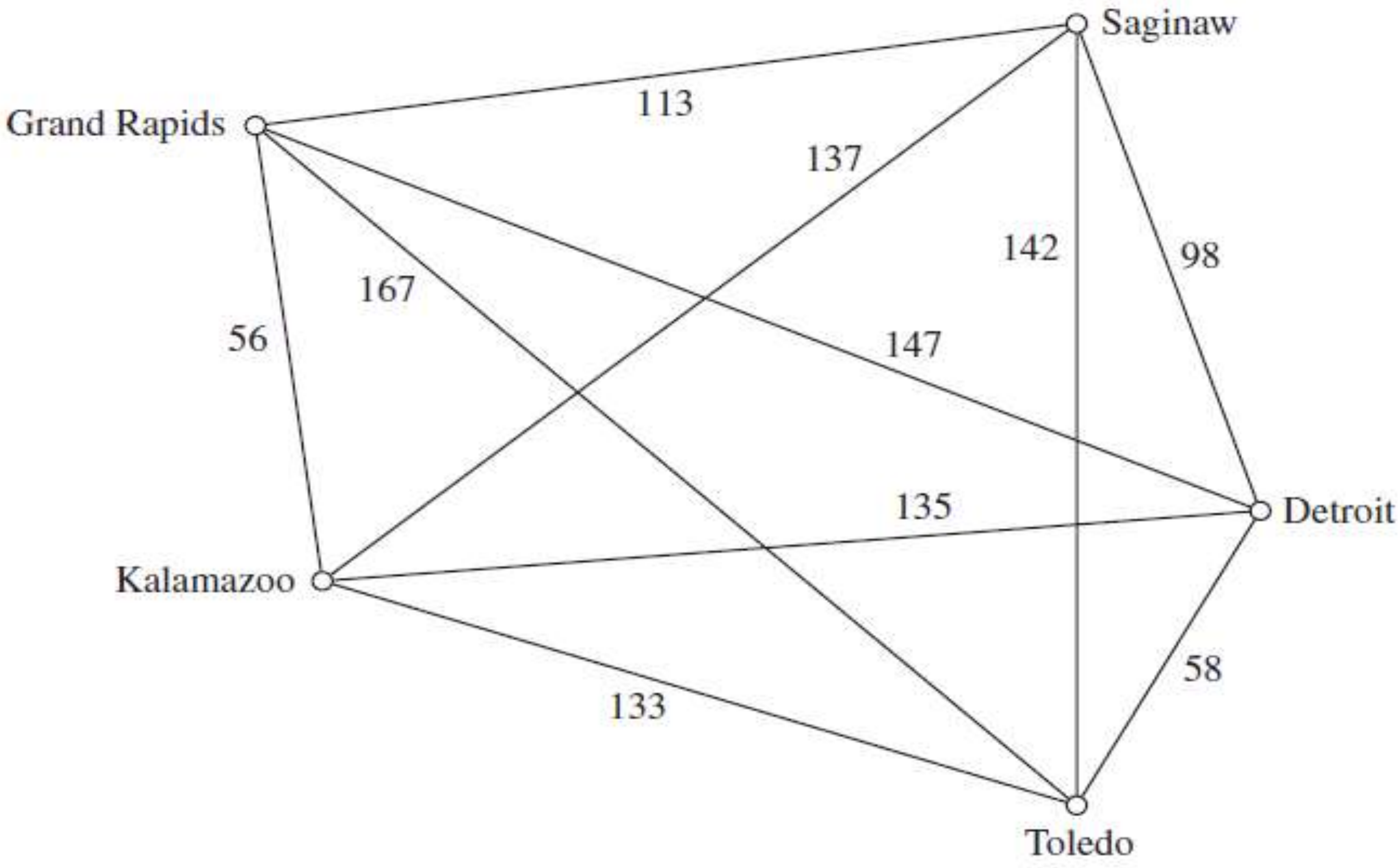


FIGURE 5 **The Graph Showing the Distances between Five Cities.**

Route	*Total Distance (miles)*
Detroit–Toledo–Grand Rapids–Saginaw–Kalamazoo–Detroit	610
Detroit–Toledo–Grand Rapids–Kalamazoo–Saginaw–Detroit	516
Detroit–Toledo–Kalamazoo–Saginaw–Grand Rapids–Detroit	588
Detroit–Toledo–Kalamazoo–Grand Rapids–Saginaw–Detroit	458
Detroit–Toledo–Saginaw–Kalamazoo–Grand Rapids–Detroit	540
Detroit–Toledo–Saginaw–Grand Rapids–Kalamazoo–Detroit	504
Detroit–Saginaw–Toledo–Grand Rapids–Kalamazoo–Detroit	598
Detroit–Saginaw–Toledo–Kalamazoo–Grand Rapids–Detroit	576
Detroit–Saginaw–Kalamazoo–Toledo–Grand Rapids–Detroit	682
Detroit–Saginaw–Grand Rapids–Toledo–Kalamazoo–Detroit	646
Detroit–Grand Rapids–Saginaw–Toledo–Kalamazoo–Detroit	670
Detroit–Grand Rapids–Toledo–Saginaw–Kalamazoo–Detroit	728

- The traveling salesperson problem asks for the circuit of minimum total weight in a weighted, complete, undirected graph that visits each vertex exactly once and returns to its starting point.
- This is equivalent to asking for a Hamilton circuit with minimum total weight in the complete graph, because each vertex is visited exactly once in the circuit.
- if there are n vertices in the graph, Once a starting point is chosen, there are $(n - 1)!$ different Hamilton circuits to examine.
- Hamilton circuit can be traveled in reverse order, we need only examine $(n - 1)!/2$ circuits to find our answer.